BANGLADESH UNIVERSITY OF BUSINESS AND TECHNOLOGY

Department of Computer Science and Engineering

Design and Implementation of an Edge-IoT and Centralized AI Facial Recognition System for Smart Access Control and Automation

Submitted By

Name	ID
Prottoy Vhattacharyya	20234103430
Chinmoy Roy Dipto	20234103449
Arifa Islam	20234103205
Suraiya Rahman Mitu	20234103215

Submitted To

Ankan Saha
Lecturer
Department of CSE
Bangladesh University of
Business and Technology (BUBT)

Date: August 17, 2026

Abstract

This report documents the design and implementation of an IoT-based smart room and gate access control system that combines face recognition, keypad PIN fallback, and remote environmental control. The system integrates an ESP32-CAM for video capture, two ESP8266 microcontrollers for access control and room automation respectively, and a Django backend that performs face detection (YOLOv8) and face recognition (DeepFace). Authorized users can unlock the gate via face verification or a PIN code and control room fan and lighting through companion web and Android applications, while administrative operations such as face enrollment, face deletion, and gate password management are restricted to an admin account.

Contents

1 Introduction

The Smart Room & Gate Access Control System is an IoT security and automation solution designed to provide contactless, face-recognition-based gate access alongside remote control of room appliances (fan and light). The system is built around three microcontroller-class devices and a central Django server:

- An **ESP32-CAM** module that streams live video for face capture.
- A **primary ESP8266** (Access Control Unit) equipped with a keypad, OLED display, and I/O expander, responsible for triggering face verification and handling PIN-based fallback authentication.
- A **secondary ESP8266** (Room Automation Unit) that drives the gate servo, fan servo, room LEDs, and a DHT11 temperature/humidity sensor.
- A **Django backend** that performs face detection using YOLOv8, face matching using DeepFace, manages user accounts, and exposes a REST API consumed by both microcontrollers as well as the web and Android applications.

The system supports two classes of users: an **admin** account with full control over the face database and gate password, and **regular users** who may open the gate via face verification and operate the fan and light, but cannot enroll or delete faces or change the gate password.

2 System Architecture

2.1 Circuit Diagram

Figure 1 shows the complete hardware wiring of the system, including both ESP8266 modules, the ESP32-CAM, the DHT11 sensor, gate and fan servos, the 4x4 matrix keypad connected via a PCF8574 I2C GPIO expander, and the OLED status display.

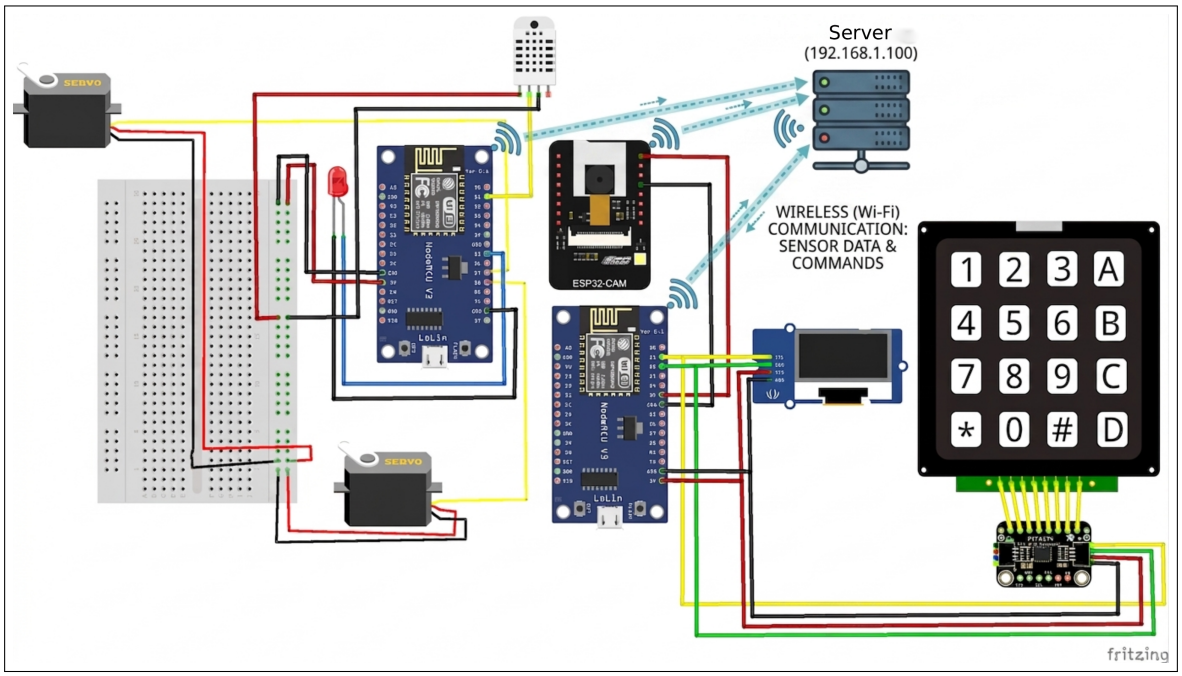

Figure 1: Complete circuit diagram of the Smart Room & Gate Access Control System, showing the two ESP8266 modules, ESP32-CAM, DHT11 sensor, gate/fan servos, OLED display, and I2C keypad with PCF8574 expander (Fritzing).

2.2 Hardware Components

Component	Role
ESP32-CAM	Captures and streams live video frames used for face detection and enrollment.
ESP8266 #1 (Access Control)	Runs the keypad interface and OLED display; authenticates with Django, triggers face verification, handles PIN fallback, and signals ESP8266 #2 to open the gate.
ESP8266 #2 (Room Automation)	Hosts a local web server exposing gate, fan, light, and sensor control endpoints; runs the auto temperature-controlled fan logic.
4x4 Matrix Keypad + PCF8574	Provides manual PIN entry via I2C GPIO expansion.
SSD1306 OLED Display	Shows system status, verification results, and PIN entry feedback.
DHT11 Sensor	Measures room temperature and humidity for automatic fan control.
Gate Servo	Physically actuates the gate lock/latch mechanism.
Fan Servo	Drives the fan speed/on-off actuator.
LEDs	Represent the room light ON/OFF states.

2.3 Communication Overview

Both ESP8266 units communicate with the Django server and with each other over the local Wi-Fi network:

- **ESP8266 #1 ↔ Django Server**: Logs in with admin credentials to obtain a session cookie, periodically syncs the gate PIN from the server, calls the `/verify/` endpoint to check a captured face, and calls `/faces/{name}/` to enroll new users.
- **ESP8266 #1 → ESP8266 #2**: On successful face verification (or correct PIN entry), ESP8266 #1 sends an HTTP **GET** request to ESP8266 #2's `/gate_open` endpoint to physically open the gate.
- **ESP8266 #2 ↔ Django Server / Web & Android Apps**: Exposes its own local REST API (fan, light, gate, sensors) which is consumed directly by the web dashboard and Android application for real-time room control, independent of the face-recognition flow.
- **ESP32-CAM → Django Server**: Streams video continuously; the Django server pulls a frame from this stream whenever a face capture is requested (enrollment or verification).

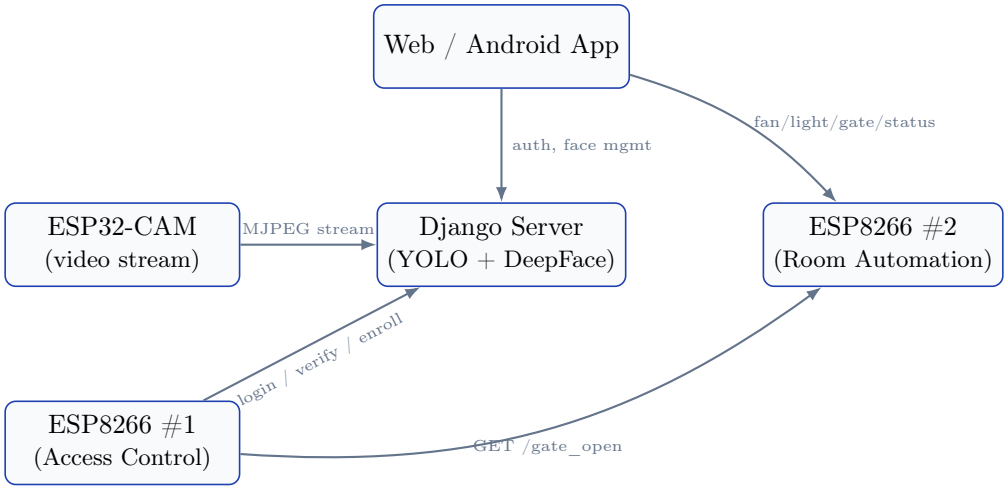

Figure 2: High-level communication flow between devices, the Django server, and client applications.

2.4 Explanation of Figure 2

Figure 2 is a simplified block diagram showing how the five main components of the system exchange data over the local network. Each box represents a device or application, and each arrow represents a distinct HTTP communication path.

2.4.1 Nodes

- **ESP32-CAM** — the camera unit that continuously streams video.
- **Django Server** — the backend running YOLOv8 (face detection) and DeepFace (face matching).
- **ESP8266 #1** — the Access Control Unit (keypad and OLED display).
- **ESP8266 #2** — the Room Automation Unit (fan, light, and gate).
- **Web / Android App** — the client applications used by end users.

2.4.2 Communication Paths

1. **ESP32-CAM → Django Server** (*MJPEG stream*): the camera continuously streams video to the server. The server pulls a single frame from this stream whenever it needs to detect or match a face, i.e. during a call to `/verify/` or `/faces/{name}/`.
2. **ESP8266 #1 → Django Server** (*login / verify / enroll*): the access-control unit logs in to obtain a session cookie, then calls `/verify/` when a user presses the **A** key on the keypad, and calls `/faces/{name}/` when enrolling a new user.
3. **ESP8266 #1 → ESP8266 #2** (*GET /gate_ open*): this is the key handoff in the security flow. Once ESP8266 #1 receives a `"success"` response from `/verify/`, or the user enters a correct PIN, it calls ESP8266 #2's `/gate_open` endpoint directly to physically unlock the gate. This connection bypasses the Django server entirely — it is a direct device-to-device call over the local network.
4. **Web/Android App → Django Server** (*auth, face mgmt*): the client applications use the server for login/signup, and for admin-only operations such as enrolling or deleting faces and updating the gate password.
5. **Web/Android App → ESP8266 #2** (*fan/light/gate*): for everyday room control, the applications talk directly to ESP8266 #2's REST API rather than routing through Django,

since fan and light control does not require face recognition.

2.4.3 Key Takeaway

The diagram illustrates that the system is organized around two largely independent control loops:

- A **security loop** — camera → server → ESP8266 #1 → ESP8266 #2 — gated by face or PIN verification.
- A **convenience loop** — client apps talking directly to ESP8266 #2 for fan and light control.

The Django server sits in the middle primarily as the *authentication and face-recognition authority*, rather than as a pass-through relay for every hardware command.

3 Django Server API Endpoints

The Django backend exposes the following REST endpoints. Face detection is performed with a fine-tuned YOLOv8 face model, and recognition against enrolled users is performed with DeepFace over a per-user image database.

GET `/`

Renders the main dashboard (`index.html`) with the camera stream URL and control server URL. Requires an active session; redirects to `/login/` otherwise.

GET `/controls/`

Renders the room controls page (`controls.html`) used to operate the fan, light, and gate via ESP8266 #2.

GET / **POST** `/login/`

Authenticates a user against the `users` table and starts a session. On **POST** with valid credentials, redirects to `/`; on failure, re-renders the login page with an error message.

GET / **POST** `/signup/`

Registers a new user account. Rejects duplicate usernames; on success, logs the new user in and redirects to `/`.

GET `/logout/`

Flushes the current session and redirects to the login page.

GET `/get_updated_password/?username={username}`

Returns the current gate PIN (`gate_password`) stored for the given user. Used by ESP8266 #1 to periodically sync the PIN used for keypad fallback authentication.

POST `/update_password/`

Admin only. Updates the gate password for the logged-in admin session. Requires the request body field `new_password`. Returns `401` if the session is missing or does not belong to the `admin` account.

POST `/faces/{name}/`

Admin only. Captures a frame from the camera stream, detects the largest/most confident face with YOLOv8, and enrolls it under the given username. Clears the cached DeepFace embeddings so the new face is included in future verification.

DELETE `/faces/{name}/`

Admin only. Deletes the enrolled face data for a specific user and clears the DeepFace cache.

DELETE `/faces/`

Admin only. Deletes *all* enrolled faces and resets the face database directory.

GET `/verify/?threshold={value}`

Captures a frame, detects a face, and matches it against all enrolled users using DeepFace. Returns the matched username and a confidence score if the match distance is within the given threshold (default `0.5`); otherwise returns a `401` failure response. This endpoint is available to any authenticated user, not just the admin, since any enrolled user may open the gate with their own face.

3.1 Access Control Summary

Endpoint	Method	Access Level
/faces/{name}/	POST, DELETE	Admin only
/faces/	DELETE	Admin only
/update_password/	POST	Admin only
/verify/	GET	Any authenticated user
/get_updated_password/	GET	Any authenticated user
/, /controls/	GET	Any authenticated user
/login/, /signup/, /logout/	GET/POST	Public

4 ESP8266 #2 (Room Automation) API Endpoints

ESP8266 #2 hosts a lightweight HTTP server on port 80 that drives the gate, fan, and light hardware, and reports live sensor readings. All endpoints return a JSON status object produced

by `getSystemStatusJson()`.

GET `/gate_open`

Opens the gate by driving the door servo to `180°`. The gate automatically closes after 5 seconds via a non-blocking timer (`checkGateAutoClose()`), without requiring a separate close request.

GET `/mode?state={auto|manual}`

Switches the system between **AUTO** mode (fan is driven automatically based on the temperature threshold) and **MANUAL** mode (fan is controlled only by explicit requests). Returns `400` if the `state` parameter is missing or invalid.

GET `/turn_on_light`

Turns the room light ON (drives the blue LED high, yellow LED low).

GET `/turn_off_light`

Turns the room light OFF (drives the blue LED low, yellow LED high).

GET `/fan?state={on|off}`

Manually sets the fan servo state. Returns `400` for a missing or invalid `state` parameter.

GET `/sensors`

Reads the DHT11 sensor and returns the current temperature and humidity. Returns `500` if the sensor read fails.

GET `/status`

Returns the full system status: current mode, light status, fan status, gate status, configured temperature threshold, and the last known temperature/humidity readings. Includes a permissive CORS header so it can be polled directly from the web dashboard.

4.1 Automatic Temperature Control

When the system is in **AUTO** mode, `checkTemperatureAutoControl()` runs every 3 seconds: the fan is switched ON once the temperature reaches the threshold (30°C) and switched OFF once it drops at least 1°C below the threshold, providing simple hysteresis to avoid rapid on/off cycling.

5 Access Control Unit (ESP8266 #1) Behaviour

ESP8266 #1 mediates between the physical keypad/OLED interface and the Django server:

1. On boot, it connects to Wi-Fi, logs in to Django with the admin credentials to obtain a session cookie, and syncs the current gate PIN via `/get_updated_password/`.

2. Pressing **A** on the keypad triggers `verify_face()`, which calls `GET /verify/`. If the response status is `"success"`, the unit calls ESP8266 #2's `/gate_open` endpoint to unlock the gate.
3. If face verification fails, the user can fall back to PIN entry: digits are collected on the keypad and, on pressing #, compared against the locally cached PIN (refreshed from the server on every attempt). A correct PIN also triggers `/gate_open` on ESP8266 #2.
4. Pressing **C** clears the currently entered PIN, and the OLED display shows masked input (`*`) as digits are typed.

6 Web and Android Applications

In addition to the physical keypad and face-recognition flow, the system is accompanied by a **web dashboard** and a companion **Android application**. Both clients talk to the same backend surfaces described above:

- **Room controls** (fan, light, gate) are sent directly to ESP8266 #2's REST API (`/turn_on_light`, `/turn_off_light`, `/fan`, `/gate_open`, `/status`) and are available to *any authenticated user*, not just the admin.
- **Face management** (enroll, delete) and the **gate password update** are routed through the Django server's `/faces/{name}/`, `/faces/`, and `/update_password/` endpoints, which are restricted to the `admin` account at the server level.
- **Face-based gate entry** is available to every enrolled user through `/verify/` — any registered face that matches within the configured threshold can open the gate, regardless of whether that account is the admin.

6.1 User Roles Summary

Capability	Admin Account	Regular User
Open gate via face recognition	Yes	Yes
Open gate via keypad PIN	Yes	Yes (shared PIN)
Control fan / light (web & Android)	Yes	Yes
Enroll a new face	Yes	No
Delete a face / reset database	Yes	No
Update the gate password	Yes	No

7 Conclusion

The Smart Room & Gate Access Control System demonstrates a practical integration of computer-vision-based access control with everyday IoT room automation. By splitting responsibilities

across a dedicated access-control ESP8266, a room-automation ESP8266, an ESP32-CAM, and a Django server that performs the heavier face detection and recognition workload, the system keeps each microcontroller's firmware simple while centralizing security-sensitive operations like face enrollment, deletion, and gate password changes behind admin-only server-side checks. The accompanying web and Android applications extend this same access model to everyday users, allowing convenient control of the fan and light while keeping database-altering operations restricted to the administrator.